

Week 3 SQL Advanced Analytics – Query Results

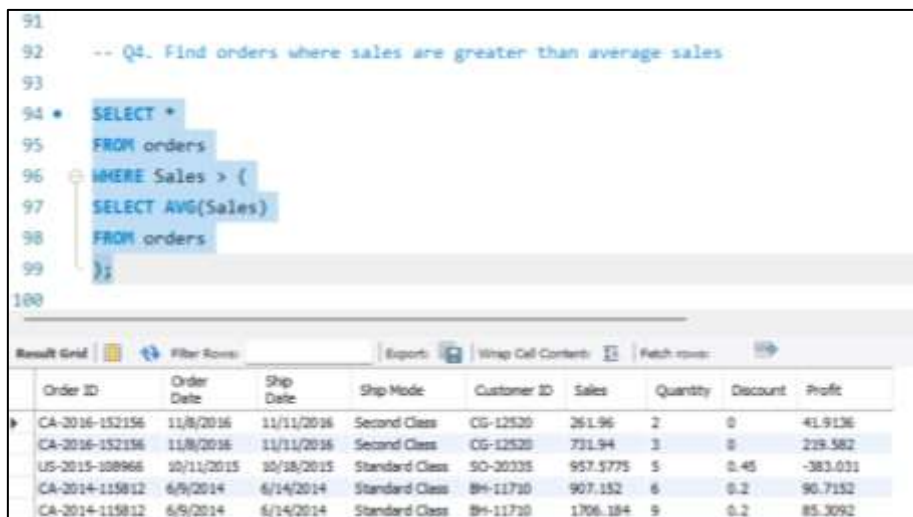
(These queries are included in the `advanced_sql_queries.sql` file.
The following queries are from Q4 to Q9 and Q11.)

1. Find all orders where sales are greater than the average sales

SQL Query:

```
SELECT *  
  
FROM orders  
  
WHERE Sales > (  
  
SELECT AVG(Sales)  
  
FROM orders  
  
);
```

Output:



```
91  
92 -- Q4. Find orders where sales are greater than average sales  
93  
94 * SELECT *  
95 FROM orders  
96 WHERE Sales > (  
97 SELECT AVG(Sales)  
98 FROM orders  
99 );  
100
```

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Sales	Quantity	Discount	Profit
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	261.96	2	0	41.9136
	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	907.152	6	0.2	90.7152
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092

2. Find the highest sales order for each customer

SQL Query:

```
SELECT  
  
`Customer ID`,  
  
MAX(Sales) AS highest_sales  
  
FROM orders  
  
GROUP BY `Customer ID`;
```

Output:

```
103
104 -- Q5. Find highest order value for each customer
105
106 • SELECT
107     `Customer ID`,
108     MAX(Sales) AS highest_sales
109 FROM orders
110 GROUP BY `Customer ID`;
111
```

Customer ID	highest_sales
CG-12520	731.94
DV-13045	721.875
SO-20335	957.5775
BH-11710	1706.184
AA-10480	479.97

3. Calculate total sales for each customer using CTE

SQL Query:

```
WITH customer_sales AS (
SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)SELECT *
FROM customer_sales;
```

Output:

```
119 • WITH customer_sales AS (
120     SELECT
121         `Customer ID`,
122         SUM(Sales) AS total_sales
123     FROM orders
124     GROUP BY `Customer ID`
125 )
126 SELECT *
127 FROM customer_sales;
```

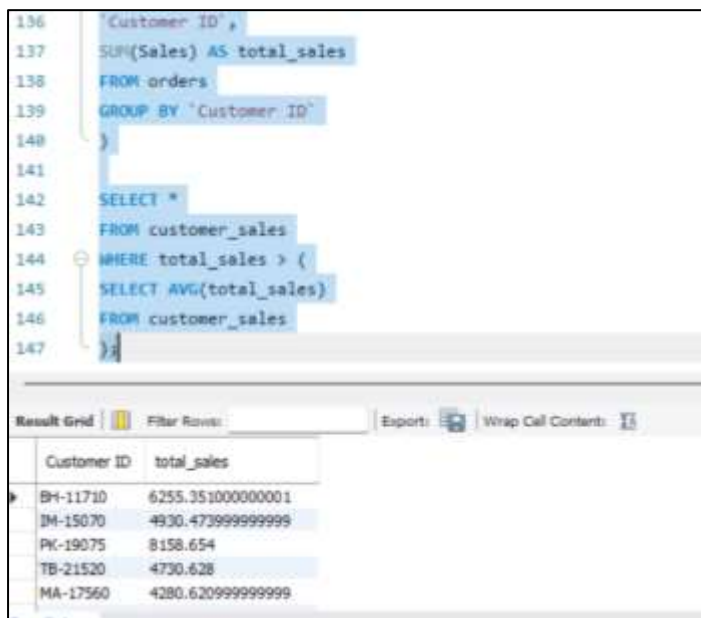
Customer ID	total_sales
CG-12520	1148.7800000000002
DV-13045	1119.4830000000002
SO-20335	2602.5750000000001
BH-11710	6255.3510000000001
AA-10480	1782.872

4. Find customers whose total sales are above average

SQL Query:

```
WITH customer_sales AS (  
    SELECT  
        `Customer ID`,  
        SUM(Sales) AS total_sales  
    FROM orders  
    GROUP BY `Customer ID`  
)  
SELECT *  
FROM customer_sales  
WHERE total_sales > (  
    SELECT AVG(total_sales)  
    FROM customer_sales  
);
```

Output:



```
136 `Customer ID`,  
137 SUM(Sales) AS total_sales  
138 FROM orders  
139 GROUP BY `Customer ID`  
140 }  
141  
142 SELECT *  
143 FROM customer_sales  
144 WHERE total_sales > (  
145     SELECT AVG(total_sales)  
146     FROM customer_sales  
147 )
```

Customer ID	total_sales
BH-11710	6255.351000000001
IM-15070	4930.479999999999
PK-19075	8158.654
TB-21520	4730.628
MA-17560	4280.620999999999

5. Rank all customers based on total sales

SQL Query:

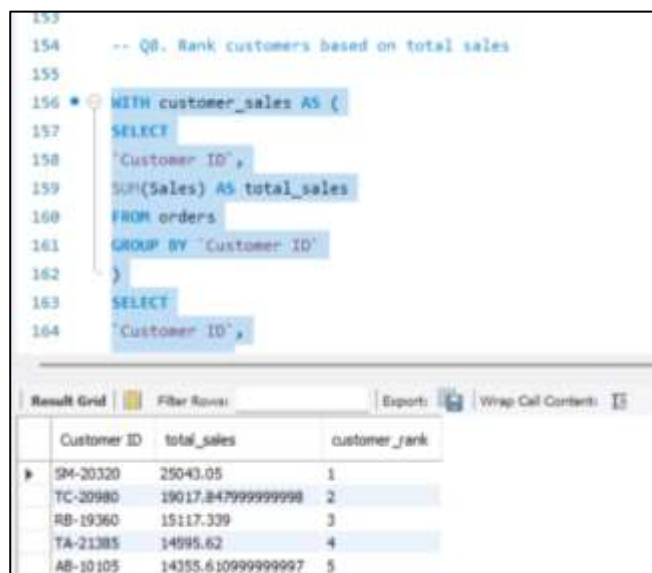
```
WITH customer_sales AS (
```

```

SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)
SELECT
`Customer ID`,
total_sales,
RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
FROM customer_sales;

```

Output:



```

153
154 -- Q8. Rank customers based on total sales
155
156 WITH customer_sales AS (
157 SELECT
158 `Customer ID`,
159 SUM(Sales) AS total_sales
160 FROM orders
161 GROUP BY `Customer ID`
162 )
163 SELECT
164 `Customer ID`,

```

Customer ID	total_sales	customer_rank
GM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5

6. Assign row numbers to each order within a customer

SQL Query:

```

SELECT
`Customer ID`,
`Order ID`,
Sales,
ROW_NUMBER() OVER (
PARTITION BY `Customer ID`

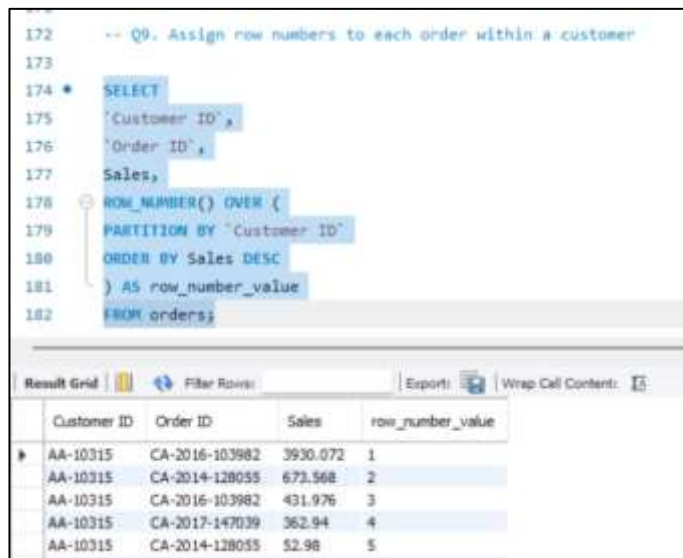
```

ORDER BY Sales DESC

) AS row_number_value

FROM orders;

Output:



```
172 -- Q9. Assign row numbers to each order within a customer
173
174 * SELECT
175   'Customer ID',
176   'Order ID',
177   Sales,
178   ROW_NUMBER() OVER (
179     PARTITION BY "Customer ID"
180     ORDER BY Sales DESC
181   ) AS row_number_value
182 FROM orders;
```

Customer ID	Order ID	Sales	row_number_value
AA-10315	CA-2016-103982	3930.072	1
AA-10315	CA-2014-128055	673.568	2
AA-10315	CA-2016-103982	431.976	3
AA-10315	CA-2017-147039	362.94	4
AA-10315	CA-2014-128055	52.98	5

7. Display top 3 customers based on total sales

SQL Query:

WITH customer_sales AS (

SELECT

`Customer ID`,

SUM(Sales) AS total_sales

FROM orders

GROUP BY `Customer ID`

)

SELECT

`Customer ID`,

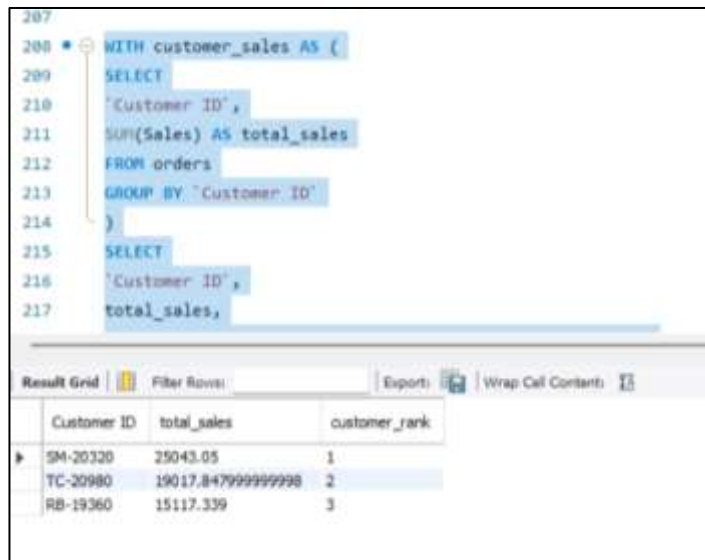
total_sales,

RANK() OVER (ORDER BY total_sales DESC) AS customer_rank

FROM customer_sales

LIMIT 3;

Output:



The screenshot shows a SQL query editor with a query starting at line 207. The query defines a CTE named 'customer_sales' and then selects from it. Below the editor, a 'Result Grid' shows the output of the query.

```
207
208 WITH customer_sales AS (
209     SELECT
210     'Customer ID',
211     SUM(Sales) AS total_sales
212     FROM orders
213     GROUP BY 'Customer ID'
214 )
215 SELECT
216 'Customer ID',
217 total_sales,
```

Customer ID	total_sales	customer_rank
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3

Final Combined Query

(Using JOIN + CTE + Window Function)

SQL Query:

```
WITH customer_sales AS (
SELECT
'Customer ID',
SUM(Sales) AS total_sales
FROM orders
GROUP BY 'Customer ID'
)
SELECT
c.`Customer Name`,
cs.total_sales,
RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
FROM customer_sales cs
```

JOIN customers c

ON cs.`Customer ID` = c.`Customer ID`;

Output:

```
228
229 * WITH customer_sales AS (
230     SELECT
231         "Customer ID",
232         SUM(Sales) AS total_sales
233     FROM orders
234     GROUP BY "Customer ID"
235 )
236 SELECT
237     c."Customer Name",
238     cs.total_sales,
239     RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: | Fetch rows:

	Customer Name	total_sales	customer_rank
▶	Sean Miller	25043.05	1
	Sean Miller	25043.05	1
	Sean Miller	25043.05	1
	Sean Miller	25043.05	1
	Sean Miller	25043.05	1